

```

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ ppt
Hello world (from Abreham Tadesse)
  0: pte 0x0000000021fcfc01 pa 0x00000000087f3f000
    0: pte 0x0000000021fcf801 pa 0x00000000087f3e000
      0: pte 0x0000000021fd005b pa 0x00000000087f40000
        1: pte 0x0000000021fcf417 pa 0x00000000087f3d000
          2: pte 0x0000000021fcf007 pa 0x00000000087f3c000
            3: pte 0x0000000021fcecd7 pa 0x00000000087f3b000
255: pte 0x0000000021fd0801 pa 0x00000000087f42000
  511: pte 0x0000000021fd0401 pa 0x00000000087f41000
    510: pte 0x0000000021fd80c7 pa 0x00000000087f60000
      511: pte 0x0000000020001c4b pa 0x00000000080007000

```

walk() function

What the function does

The walk function is used to get the physical address that a virtual address is mapped to. It does this by using the 27 bits from the virtual address, 9 bits at a time, to walk down the hierarchical page structure. The upper 9 bits direct walk() to access the specific index at the root page table.

PTE_V and PTE2PA()

If an index is valid, as indicated by a bitwise comparison with PTE_V which has a value of 1, then the entry at the specified index is used as a physical address to the next pagetable page. Conversion from the page table entry to a physical address is performed by the PTE2PA() macro. Then the process is repeated to the newly found pagetable, i.e. using the later 9 bits of the virtual address as an index into the new pagetable. This process is repeated until the last-level-down pagetable page gives us the address of the page that the virtual memory is mapped to. If at any point in this process a page fault occurs, i.e. PTE_V is 0, the required page is allocated into memory and puts the physical address of that new page into the pointing pagetable's PTE. This involves using the PA2PTE() macro to convert the given physical address into a page table entry, which involves shifting the physical address 12 bits to the right to make room for accessing the indexes within the new page, and then shifting it 10 bits to the left to make space for the flag bits.

xv6 process memory map

In xv6 a process is allocated memory in the form of a virtual address space. This gives the process the illusion that it has a large contiguous block of memory it can access but in reality these virtual addresses map to some place in memory that's not contiguous. This mapping is done in conjunction with the memory mapping hardware and the OS.

A process is allocated physical memory in forms of pages. xv6 uses a hierarchical page structure in which a single page has 512 Page Table Entries (PTE). Each entry points to a lower level page, which eventually becomes the page containing the information to be accessed. Each PTE has 8 bytes of information in which bits 10 - 53 contain the physical page number and the lower 11 bits are flags about the page being accessed.

To index into these pages xv6 uses 9 bits from the virtual address starting from bit position 12. So 27 bits in total from the virtual address are used. To index into the root page the upper 9 bits are used, into the second level page the middle 9 bits are used and the lower 9 bits are used to index into the last page.